DO NOW — RECALL

Hash tables with arrays

Name: _____ Class: _____ Date: _____

1. In a binary search tree, where are values smaller and larger than a node stored?

2. What does -1 represent in our pointer arrays?

3. What is a collision in hashing?

4. For table size 5, what bucket does key 17 map to using key MOD 5?

DO NOW — CHECK / OBJECTIVES

Correct and improve

Check page 1 in GREEN PEN. Write corrections in full.

1. Model answer

Smaller values go left and larger values go right.

2. Model answer

null / no child.

3. Model answer

Two keys map to the same hash location.

4. Model answer

2.

By the end, you can…

• explain how a hash function maps keys to buckets

• handle collisions using fixed array buckets

• create, traverse, add, search for and remove hashed data

Vocabulary — know these terms

Recall the term — hash table

Recall the term — key

Recall the term — hash function

Recall the term — bucket

Recall the term — collision

Recall the term — chaining

Recall the term — lookup

Recall the term — load

NEW LEARNING

Read, annotate and explain

Hash chooses the row; collision uses another slot

table size = 5
hash(key) = key MOD 5

12 → row 2
17 → row 2
22 → row 2

bucket 2: [12, 17, 22]

A fixed 2D array lets us model separate chaining without introducing OO or library collection classes.

Create fixed buckets

Each row is a bucket with three available collision slots.

Hash

The same key maps to the same row.

Add

A collision goes into the next free column in that bucket.

Search/remove

Only the calculated bucket is searched. Removal can shift later bucket items left.

Explain one step in your own words, using the correct technical terms.

I DO — WORKED EXAMPLE

Follow the worked method

Add 12, 17 and 22

12

```
table[2,0] = 12
count[2] = 1
```

17

```
table[2,1] = 17
count[2] = 2
```

22

```
table[2,2] = 22
count[2] = 3
```

17

```
hash → bucket 2
scan row 2
17 found at column 1
```


EXIT TICKET / REFLECTION

Check your understanding

Complete the actual lesson Exit Ticket before checking the answer.

A hash function tells you which bucket to search; it does not guarantee that only one key will use that bucket.

How confident are you? Circle one: Confident / Getting there / Need help

Explain your choice, identifying one concept you can now use independently.

GREEN PEN — Correct and explain

Choose one mistake from your independent work. Write the correct method and explain precisely why your original answer was wrong.

Next step: revisit the model or a relevant revision-guide section if needed.

YOU DO — INDEPENDENT PRACTICE

Questions · Part 2

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 3

Search only the correct bucket
Search for key 17 using its hash value and a loop across that bucket.

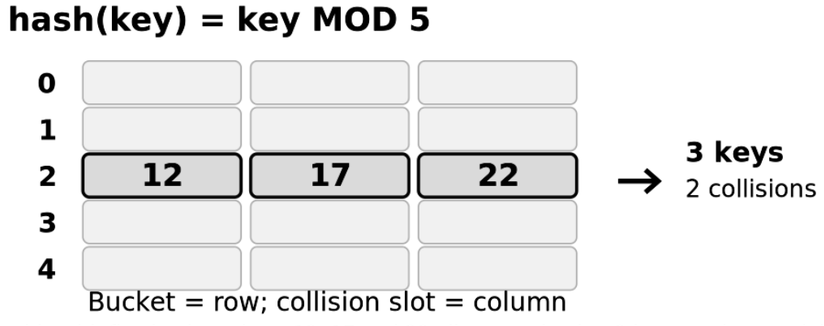
Question 4

Remove 17
Find 17 in its bucket, shift later values left, and reduce the bucket count.

I Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Add 12, 17 and 22
All three keys hash to bucket 2.



Hash table with five buckets; keys 12, 17 and 22 all map to bucket 2 because key modulo 5 is 2

WE DO — GUIDED PRACTICE

Try → Discuss → Improve

1. Calculate hash

Table size 10; key 37.

```
37 MOD 10 = ?
```

Method / working:

2. Add key

Use bucket and count arrays.

```
int bucket = key % ?;
table[bucket, ?] = key;
count[bucket]++;
```

Method / working:

3. Collision

12 and 17 both map to bucket 2.

```
bucket 2:
column 0 = 12
column 1 = ?
```

Method / working:

4. Remove from bucket

Remove column 1 when count is 3.

```
table[b,1] = table[b,?];
count[b]--;
```

Method / working:

YOU DO — INDEPENDENT PRACTICE

Questions · Part 1

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 1

Create buckets
Create a 5×3 integer table and a count array.

Question 2

Add keys with collisions
Add 12, 17, 22 and 9 using key MOD 5.